Mips Single-Cycle

- Proiectul a fost testat integral pe placa de dezvoltare Nexys.
- Funcționalitatea a fost verificată pentru toate combinațiile de switch-uri (SW 7:5), iar valorile afișate pe SSD corespund cu trasarea teoretică.
- Nu s-au efectuat simulări software suplimentare.

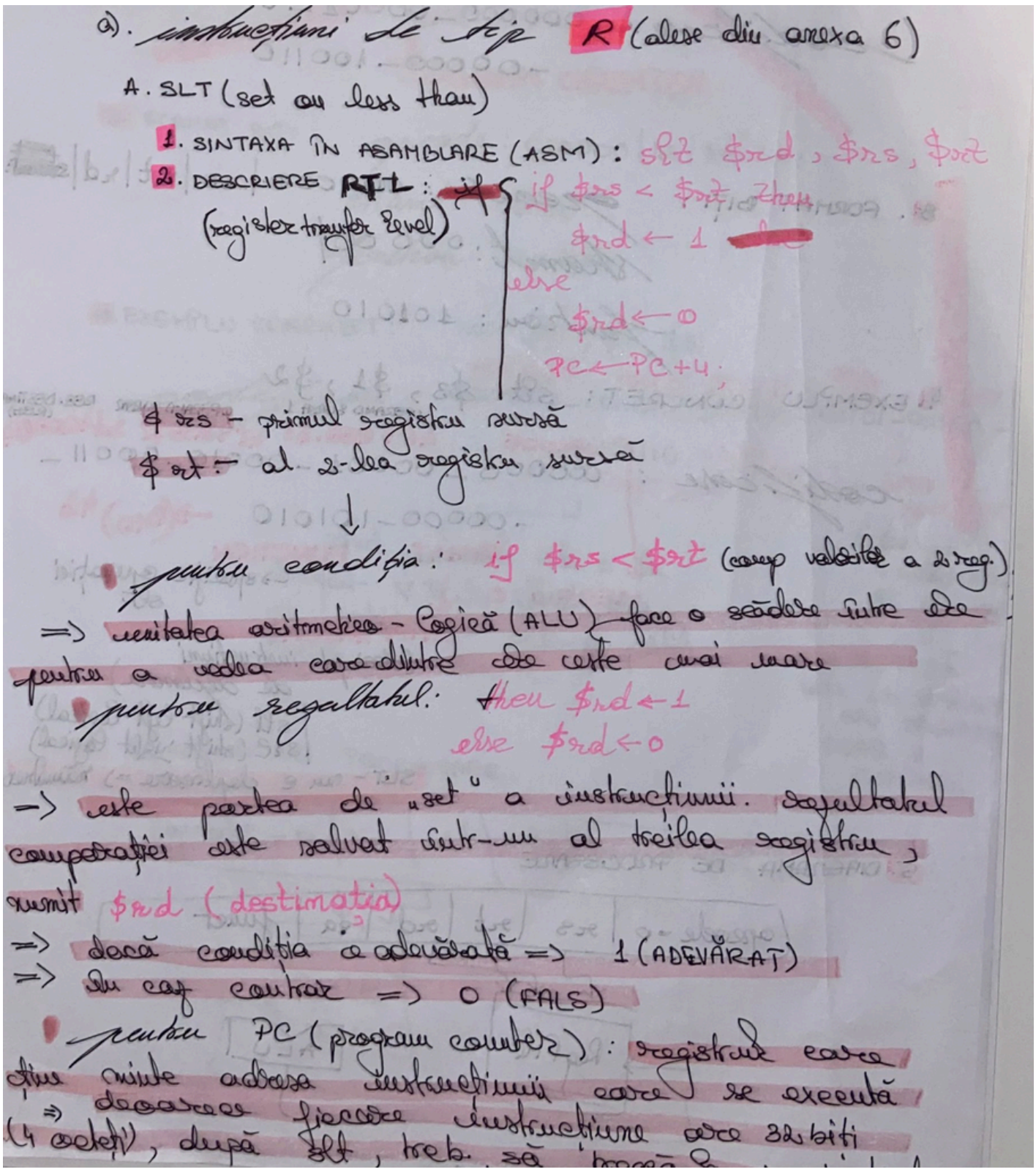
---

precizarea elementelor functionale

- **IFetch (Instruction Fetch):** Responsabil cu aducerea instrucțiunii din ROM și actualizarea PC-ului.
- **ID (Instruction Decode):** Extrage opcode-ul, registrele și valoarea imediată; conține blocul de registre (Register File).
- **EX (Execute):** Realizează operațiile aritmetice și logice prin intermediul ALU.
- **MEM (Memory):** Gestionează accesul la memoria de date RAM pentru instrucțiunile de tip Load/Store.
- **UC (Control Unit):** Generează semnalele de control necesare fiecărei instrucțiuni pe baza opcode-ului.

---

descrierea celor 4 instructiuni



3. FORMAT BIȚI: opcode | rs | rt | rd | shamt | funcție

shamt: 000 001

funcție: 101010

4. EXEMPLU CONCRET: sll $3, $1, $2

OPERAND SURSĂ 1 — OPERAND SURSĂ 2 — REG. DESTINAȚIE (REZ)

codificare: 000000_00001_00010_00011_00000_101010 → (010) 42

OPCODE → rs → REG. $1 | rt → REG $2 | rd → REG $3

SHAMT — FUNCTION (↳ specific operației sll)

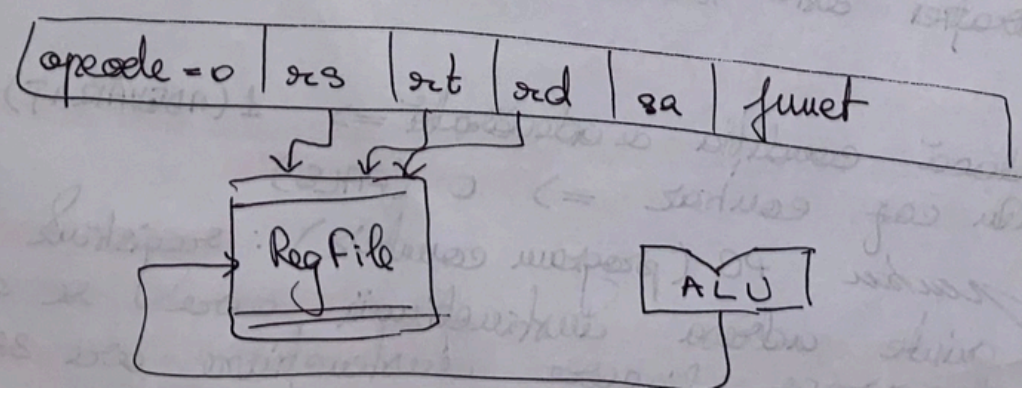
MEREU 0 ↓

SHIFT AMOUNT (doar pt. instrucțiuni de deplasare)

| sll (shift left logical)
| srl (shift right logical)

SLT - nu e deplasare => rămâne 0

5. DIAGRAMA DE PROCESARE



B. XOR (EXCLUSIVE XOR)

1. SINTAXA ASM: xor $rd, $rs, $rt

2. DESCRIERE RTL: $rd ← $rs ⊕ $rt ;
PC ← PC+4 ;
PROGRAM COUNTER

3. FORMAT BITI:
opcode: 000000 | rs | rt | rd |
shamt: 00000
function: 100110

4. EXEMPLU CONCRET: xor $5, $1, $2

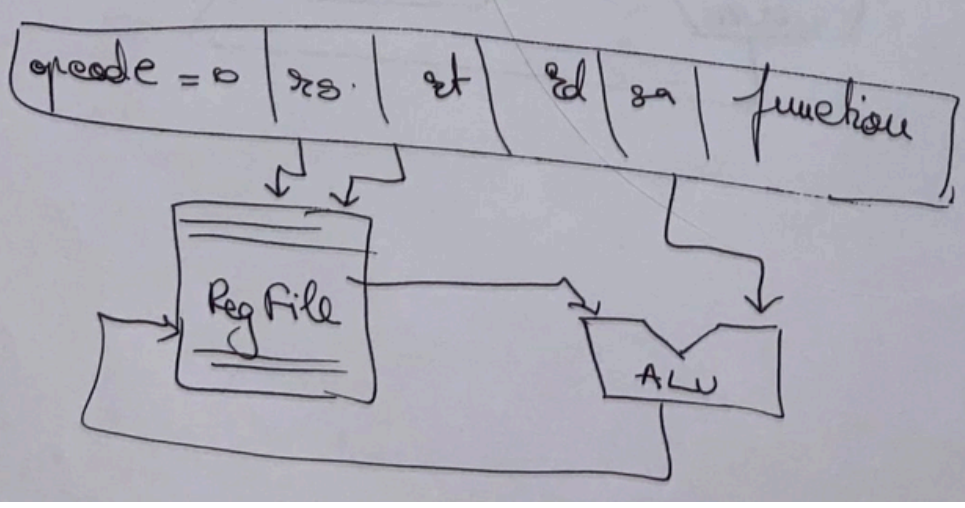
codificare: 000000 _ 00001 _ 00010 _ 00101 _ 00000 _
OPCODE REG $1 REG $2 REG. DEST SHAMT
_ 100110 → 38 (B16)
FUNCTION

când procesorul vede 0 la început și 38 la sfârșit, știe că trebuie să activeze porțile logice XOR din ALU

SHIFT AMOUNT
(doar pt. instrucțiuni de deplasare)

XOR NU MUTĂ BIȚII LA STÂNGA SAU DREAPTA DOAR ȘI COMPARĂ

5. DIAGRAMA DE PROCESARE



b). instrucțiuni de tip I (alse din anexa 6)

A. ANDI - (AND Immediate)

1. SINTAXA ASM: andi $rt, $rs, imm

2. DESCRIERE RTL: $rt ← $rs & ZE(imm);
PC ← PC+4;

op. logică AND între val. reg $1 și nr. 15

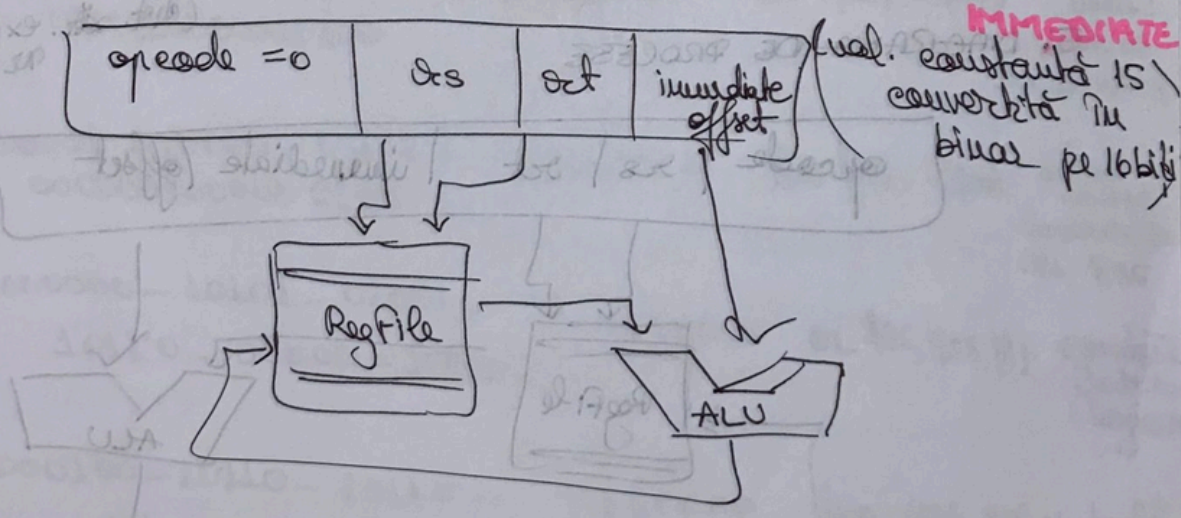
ZE (zero extension)

3. FORMAT BIȚI: opcode: 001100 | rs/at | Immediate (16 biți)

4. EXEMPLU CONCRET: andi $2, $1, 15

Codificare: OPCODE 001100 - RS 00001 - RT 00010 - 0000000000001111

5. DIAGRAMA DE PROCESARE:



B. BNE (Branch on not Equal)

EXTINS PE 32 BIȚI/INMULȚIT CU 4 (DEPLASAT LA STÂNGA CU 2 POZ) → 4 OCTEȚI

1. SINTAXA ASM: bne $rs, $rt, offset

2. DESCRIERE RTL: if $rs != $rt then
PC ← (PC+4) + (SE(offset<<2))
else PC ← PC+4;

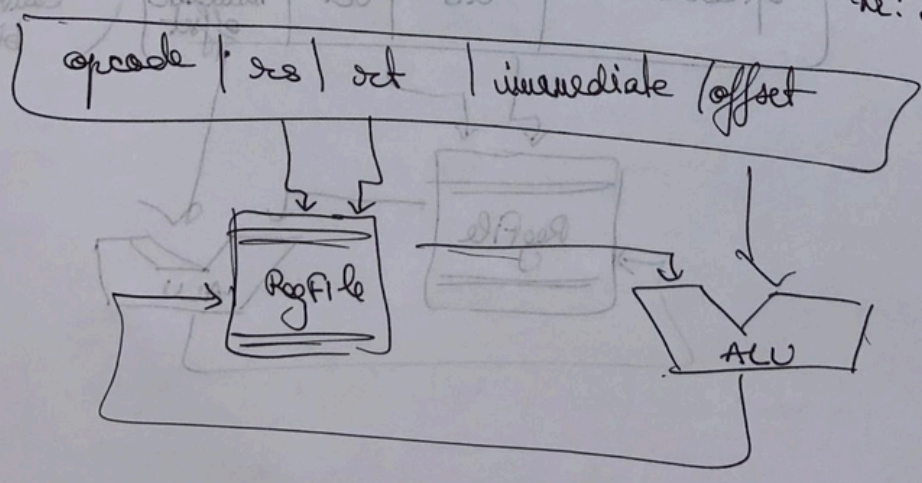
3. FORMAT BIȚI: opcode | 000101 | rs | rt | offset (16 biți)

4. EXEMPLU CONCRET: bne $1, $2, 4 (sare peste instrucțiuni dacă nu sunt egale)

codificare: OPCODE 000101 (5) | RS 00001 (1) | RT 00010 (2) | 0000000000000100 (10) (4)

OFFSET (dist. să. exprimată în nr. de instrucțiuni)

5. DIAGRAMA DE PROCESE



valorile semnalelor de control

instructiune	RegDst	ExtOp	Alu src
SLT	1	X	0
XOR	1	X	0
ANDI	0	0	1
BNE	X	1	0

Leonte Patricia Mirabela